



KelasaGaara (ಕೆಲಸಗಾರ)

Academic Project Report & Viva Defense Guide

Platform: React (Vite), Firebase Firestore, RTDB, Leaflet JS

Repository: github.com/anandaram240-wq/Kelasagara

Deployment: kelasagara-f127a.web.app

Developer: Anandaram

Session: 2026 Academic Evaluation

KelasaGaara (ಕೆಲಸಗಾರ) – Academic Project Report & Viva Defense Guide

This document is compiled as a detailed, structured, and easy-to-understand reference for your project panel. It contains a full technical report, workflow diagrams, database structures, and exactly 25 high-yield viva questions.

Part 1: Project Report

1. Introduction

KelasaGaara (ಕೆಲಸಗಾರ) is a hyper-local, bilingual (English and Kannada) service marketplace application developed for blue-collar daily-wage workers in Karnataka. The application organizes the informal job sector by enabling direct hiring, chat communication, and GPS map coordinates, eliminating middlemen and agency commissions.

2. Problem Statement

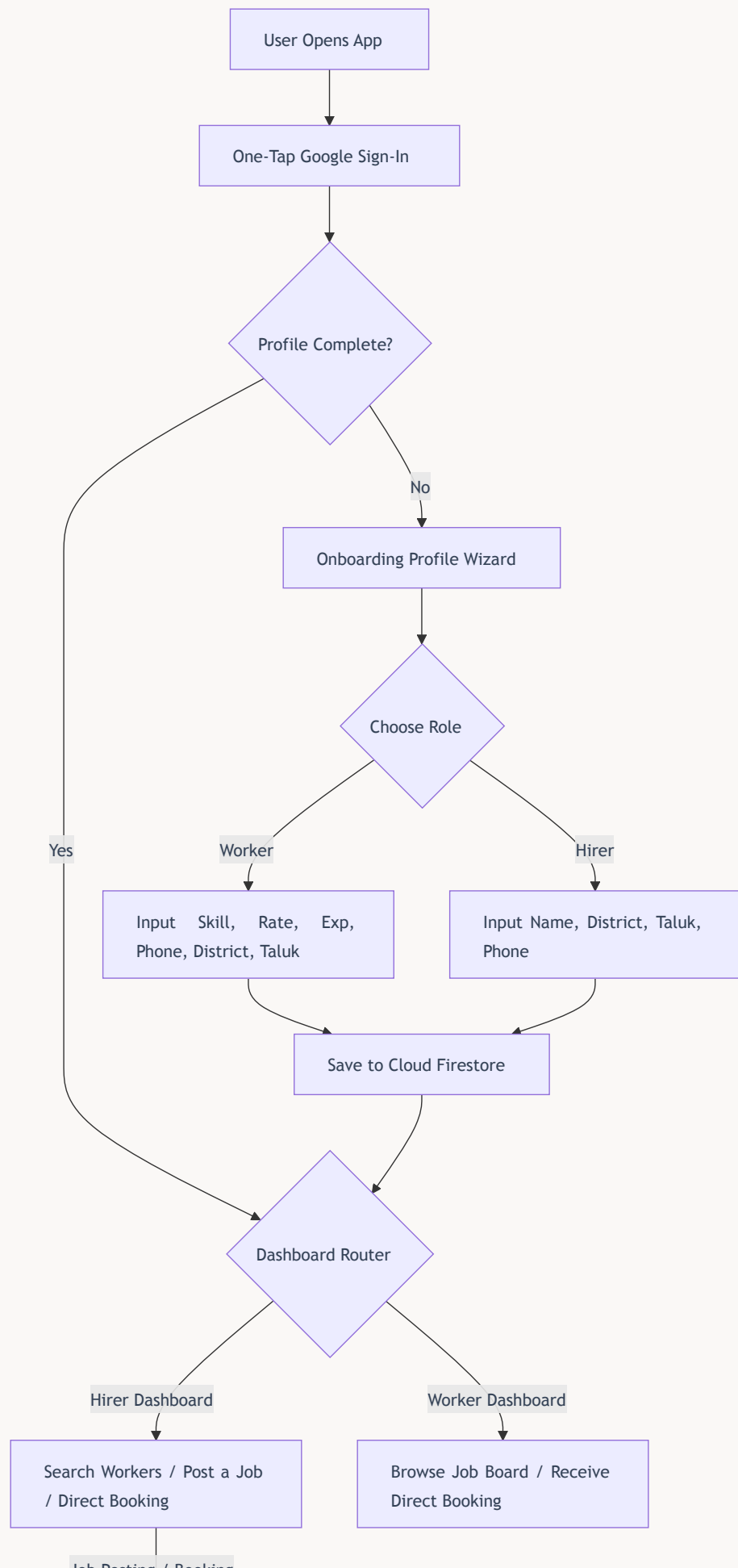
The blue-collar and daily-wage hiring market in India, specifically in Karnataka, is highly fragmented. - **Workers** (Masons, Plumbers, Electricians, Carpenters, Painters, etc.) have no digital presence and rely on physical assembly at labor stands. - **Hirers** face issues locating reliable service providers, negotiating wages, and communicating exact project addresses in rural or semi-urban taluks. - **Existing platforms** (like LinkedIn or Indeed) are built for formal white-collar jobs. Commercial home service directories (like Urban Company) charge high commissions (20% to 30%) and do not support small towns or rural areas.

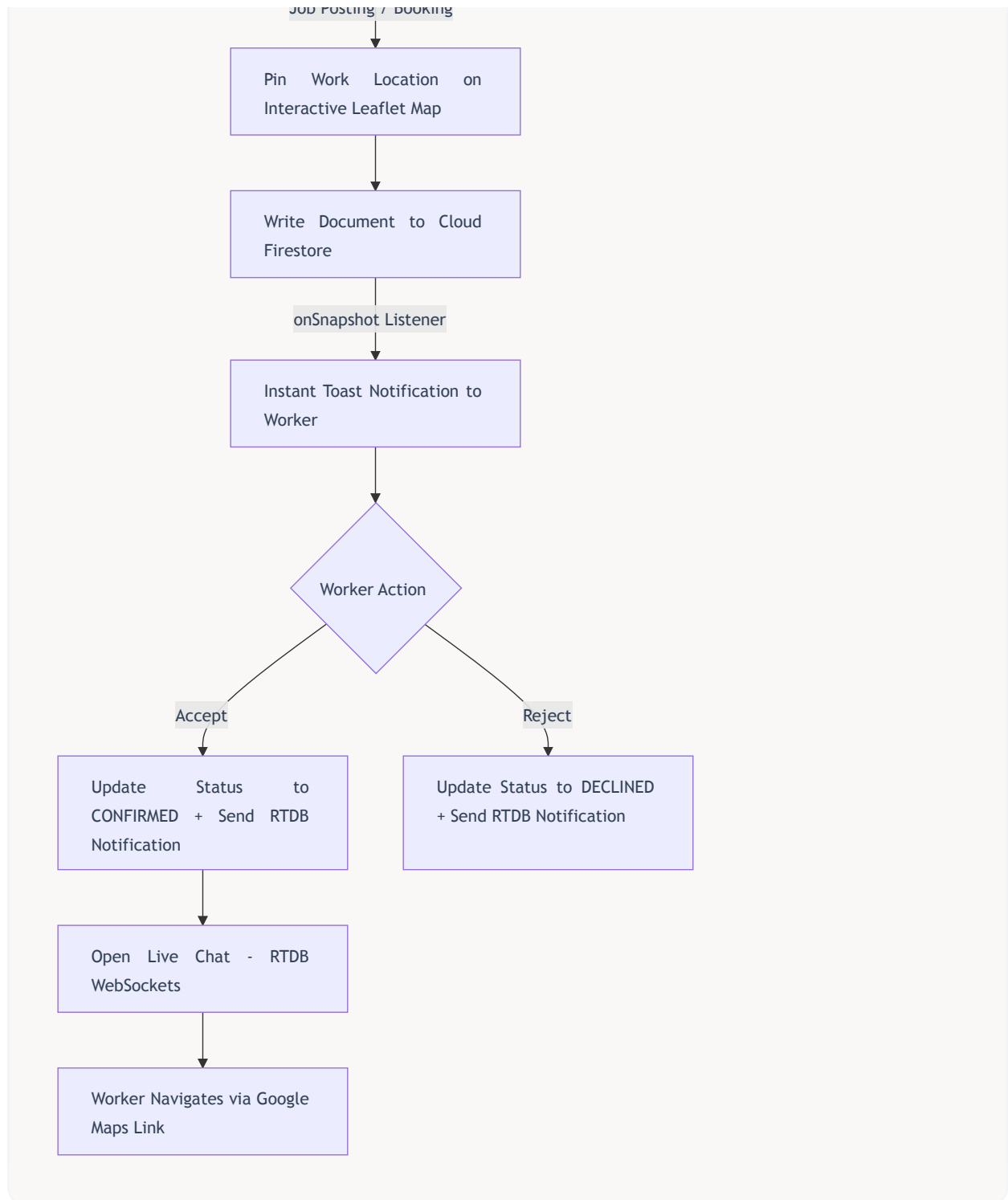
3. Objectives

- **Zero-Commission Ecosystem:** To facilitate direct worker-to-hirer connection.
- **Bilingual Interface:** Support English and Kannada natively across all pages, categories, and inputs.
- **Bypass passwords:** Utilize one-tap Google Sign-In to increase accessibility for users.
- **Interactive Location Pinning:** Integrate a map canvas so hirers can pin exact work coordinates instead of typing long, ambiguous addresses.
- **Instant Synchronization:** Establish low-latency WebSocket connections for real-time messaging, typing indicators, and immediate push alerts.

4. Proposed System Workflow

The workflow supports two separate user journeys: **Hirers** and **Workers**.





5. Detailed Technology Stack (Why We Used It)

1. Frontend: React (SPA)

2. *Why*: Allows building a Single Page Application with reusable, component-based architecture. Fast DOM rendering keeps the app lightweight for mobile phones with slow internet connections.

3. Build Tool: Vite

4. *Why*: Unlike Webpack, Vite uses native browser ES modules (ESM) to deliver fast Hot Module Replacement (HMR) and instantaneous build speeds.

5. Database 1: Cloud Firestore (NoSQL Document Store)

6. *Why*: Designed for hierarchical document queries. We use it to filter workers by trade, district, and taluk, and perform quick searches on job listings.

7. Database 2: Firebase Realtime Database (RTDB)

8. *Why*: Built on persistent WebSocket connections. It keeps database read/write latency below 100ms, which is critical for instant chat messages and real-time alerts.

9. Authentication: Firebase Auth (Google Sign-In)

10. *Why*: Blue-collar workers often forget text passwords or usernames. Google Sign-In provides one-tap passwordless security.

11. State Manager: Zustand

12. *Why*: A lightweight state store. It updates components dynamically without causing the unnecessary page re-renders associated with React Context.

13. Maps Rendering: Leaflet JS (via CDN)

14. *Why*: An open-source mapping library. It bypasses expensive Google Maps JavaScript SDK keys, keeping project hosting completely free.

15. Geocoding Engine: OpenStreetMap Nominatim API

16. *Why*: Converts latitude and longitude coordinates into actual street addresses (and vice versa) for free.

17. Animations: Framer Motion

18. *Why*: Used to build smooth modal overlays, tab switches, and slide-in alerts.

Part 3: Data Storage Details (Where & What)

We divide the data into two databases to optimize read/write costs and messaging speeds:

DATABASE	DATA TYPE	COLLECTIONS / PATHS	PROPERTIES STORED
Cloud Firestore	Persistent Structured Data	<code>users/{uid}</code>	User profiles, role, skills, daily rate, district, taluk, phone.
Cloud Firestore	Queryable Jobs	<code>jobs/{jobId}</code>	Job title, description, skills required, district, taluk, pay, coordinates.
Cloud Firestore	Booking Agreements	<code>bookings/{bookingId}</code>	Direct bookings, date, time, duration, status, payment terms, location coordinates.
Realtime DB	Chat messages	<code>chats/{bookingId}/messages</code>	Message text, sender/receiver UIDs, timestamp, read status.
Realtime DB	Presence Tracking	<code>presence/{userId}</code>	Online status (boolean), last active timestamp.
Realtime DB	Real-time Alerts	<code>notifications/{userId}</code>	Live toast notifications for incoming requests.

Part 4: 25 High-Yield Viva Questions & Answers

1. Database & Backend Architecture

Q1: Where is the user database stored?

The user database is stored in **Cloud Firestore** (NoSQL Document Database) under the **users** collection. Each user's document uses their unique Google Authentication UID as the document ID.

Q2: What is the difference between Cloud Firestore and Firebase Realtime Database?

Cloud Firestore is a document-based database designed for complex queries, filtering, and sorting (like searching workers by district/taluk). The Realtime Database is a single JSON tree built on direct WebSockets, optimized for instant, low-latency updates like chat messaging and presence indicators.

Q3: How are chat messages stored and structured?

Chat messages are stored in the **Firebase Realtime Database** under the **chats/{bookingId}/messages** path. Storing them here allows messages to sync between both clients in under 100ms via persistent WebSocket channels.

Q4: How do you prevent unauthorized users from editing other users' data?

We write **Firestore Security Rules**. We enforce validation checks like `allow write: if request.auth != null && request.auth.uid == userId;`. This blocks requests if the authenticated user's ID does not match the target document ID.

Q5: How does the database track if a user is online or offline?

We write to `presence/{userId}` in the **Realtime Database**. When the user opens the app, we write `{ online: true }`. If they close the tab, Firebase's `onDisconnect()` hook automatically updates the database to `{ online: false }` on the server side.

2. Frontend & React Core

Q6: What is a Single Page Application (SPA), and how is it routed?

An SPA loads a single HTML page and dynamically updates the DOM as the user interacts with the app. We use **React Router** to handle routing. In production, we add a rewrite rule in `firebase.json` so that all server URLs redirect to `index.html`, allowing the client router to render the correct view.

Q7: Why did you use Zustand instead of Redux?

Zustand has a minimal bundle size and zero boilerplate. Redux requires actions, reducers, and store configurations, which increases bundle size. Zustand also prevents unnecessary re-renders of parent components, keeping performance high on mobile devices.

Q8: What is the difference between Vite and Webpack?

Webpack bundles the entire project source code before serving it, which slows down development servers. Vite uses native browser ES Modules (ESM) to serve files on-demand and applies Hot Module Replacement (HMR) instantly.

Q9: How is the user login session preserved when the browser is refreshed?

The Firebase Auth SDK stores the user's login tokens in the browser's **IndexedDB** cache. Upon reload, the SDK reads this cached session and restores the user context.

Q10: What are React hooks, and which ones did you use?

Hooks are functions that let you tap into React state and lifecycle features from functional components. We used `useState` (local state), `useEffect` (side effects and listeners), `useRef` (DOM elements reference), and custom hooks like `useAuth` to bind global states.

3. Maps, Location & Geocoding

Q11: How do you show maps without a paid Google Maps API key?

We load the **Leaflet JS** library from a CDN. Leaflet renders free map tiles from OpenStreetMap (OSM) on a standard HTML canvas element. This keeps map rendering free.

Q12: How does a user select a location on the map?

We bind click/tap event listeners to the Leaflet map container. When a user taps or drags the marker pin, we capture the geographical coordinates (Latitude and Longitude) and update the form state.

Q13: How do you convert coordinates to a readable address?

We send a request to the free **OpenStreetMap Nominatim Reverse Geocoding API** using the coordinates. The API returns a JSON response containing the matching street, taluk, district, and pincode.

Q14: How does the "Use Current GPS Location" button function?

It calls the browser's native `navigator.geolocation.getCurrentPosition` API. Once the browser returns the user's coordinates, we pan the Leaflet map center to those coordinates and trigger a reverse-geocoding call.

Q15: How do you handle cases where the user denies browser location permissions?

We catch the permission error callback. The app displays an informative error toast notifying the user to grant permissions, and allows them to search or type the address manually to ensure the workflow is not blocked.

4. Direct Bookings & Notifications

Q16: How do workers receive notifications instantly without refreshing?

We use Firestore's `onSnapshot` listener on the `bookings` collection. When a new booking with a matching `workerId` is added to Firestore, the database pushes the new document to the active client, which displays a toast alert.

Q17: What happens when a worker clicks 'Accept' on a booking?

The app calls `updateDoc` to change the booking status in Firestore to `CONFIRMED`. It then calls `sendNotification` to write an alert in the Realtime Database under the hirer's UID, which displays a confirmation on the hirer's dashboard.

Q18: How does the worker navigate to the job location?

We save a structured `workLocation` object containing a pre-formatted Google Maps URL (`https://www.google.com/maps?q=lat,lng`). On the worker's dashboard, this renders a button that opens Google Maps directly with navigation.

Q19: How are unread chat counts maintained?

Inside our Zustand `useChatStore`, we listen to new children added in RTDB. If a message arrives for a thread that is not currently open, we increment the unread count in state and write it locally.

Q20: How did you implement bilingual support (English/Kannada)?

We maintain a local translation dictionary matching UI labels. A state variable (`lang`) toggles between 'en' and 'kn'. Key static arrays (like the list of trades or districts) are loaded based on the current active language.

Q21: How do you validate form fields before submitting a booking?

Before calling the Firestore `addDoc` function, we check the form state values (e.g. `if (!form.jobTitle || !form.date) ...`). If empty, we trigger a Toast warning and block the database write.

Q22: Why did you choose NoSQL instead of SQL for this project?

A NoSQL structure allows us to scale out horizontally easily. Since chat messages, notifications, and user profiles do not follow a strict tabular structure, NoSQL lets us store records as flexible documents and JSON trees.

Q23: How do you handle spelling differences in Karnataka district names (e.g. 'Mysore' vs 'Mysuru')?

We implemented an `aliasMap` in our geocoding function. If OpenStreetMap returns historical names (like 'Gulbarga' or 'Belgaum'), our system translates them to the official names ('Kalaburagi', 'Belagavi') before matching them against our 31-district database.

Q24: What is the role of `firebase.json` ?

It configures Firebase Hosting. It directs Firebase to serve assets from the `dist` build directory, and contains a rewrite rule `{ "source": "**", "destination": "/index.html" }` which redirects all incoming URLs to `index.html` so that our React Router client-side routing works without 404 errors.

Q25: What is the purpose of the `dist` directory in your React project?

The `dist` (distribution) directory contains the production-ready build of the application. Vite compiles, minifies, and bundles all React JSX, CSS variables, and JS modules into static assets inside this folder for deployment to hosting servers.